Graph Theory Foundations: Dijkstra's Single-Source Shortest Path Algorithm, Topological Sorting in Directed Acyclic Graphs, and Queue-Heap State Dynamics: A Formal Algorithmic Analysis and Systems Engineering Compendium

Abstract

This research treatise delivers a comprehensive theoretical formalization and production-grade engineering analysis of two foundational pillars of graph theory: Dijkstra's Single-Source Shortest Path (SSSP) Algorithm and Topological Sorting in Directed Acyclic Graphs (DAGs). We bridge the pedagogical continuum from elementary unweighted Breadth-First Search (BFS) to priority-driven exploration, systematically deconstructing: (1) The "Another BFS" paradigm—edge-splitting via auxiliary dummy vertices for bounded integer weights ($w \in \{1, 2\}$) and the mathematical proof of why arbitrary weight bounds ($W \geq 10^5$) trigger catastrophic time and memory bloat, necessitating priority-driven state spaces; (2) The greedy choice property and inductive optimality proof of Dijkstra's algorithm, establishing why non-negative edge constraints ($w(u, v) \geq 0$) are indispensable and dissecting the classical 3-node negative edge failure mode; (3) Priority queue mechanics in modern runtime systems (specifically the Java Virtual Machine), contrasting theoretical $O(V \log V + E)$ Fibonacci heaps against industrial lazy-deletion binary min-heaps, and proving the asymptotic $O((V + E) \log V)$ runtime driven by the essential stale-element pruning guard; (4) Topological sorting across DAGs, proving the existence theorem, Hamiltonian path uniqueness criterion, and cycle detection mechanics; (5) The algorithmic duality between Kahn's In-Degree Reduction Algorithm (source-driven BFS) and Post-Order Finish-Time DFS (sink-driven exploration with 3-color state cycles); (6) Rigorous, step-by-step state-space execution traces replicating every lecture dry run, heap state, and quiz problem; and (7) Industrial, bug-free Java reference architectures complete with path reconstruction and cycle extraction.

Index Terms

Graph Theory, Dijkstra's Algorithm, Shortest Path Optimality, Topological Sort, Directed Acyclic Graph (DAG), Kahn's Algorithm, In-Degree Reduction, Depth-First Search, PriorityQueue, Lazy Deletion, Edge Relaxation, Greedy Choice Property, Cycle Detection, Computational Complexity, Java.

1 Introduction and the Spectrum of Shortest Path Exploration

In computational graph theory, discovering the optimal trajectory between vertices under varying edge weight metrics represents a fundamental challenge in computer systems, routing protocols, distributed consensus, and compilation dependency resolution. The algorithmic mechanism required to resolve this problem depends entirely on the algebraic properties of the edge weight function $w : E \to \mathbb{R}$.

1.1 The Uniform Baseline: Unweighted BFS

Let $G = (V, E)$ be an unweighted, connected graph. Breadth-First Search (BFS) explores the state space in concentric wavefronts using a First-In, First-Out (FIFO) queue Q. Because each edge transition carries an identical unit traversal cost ($w(e) = 1$), the queue maintains a monotonic invariant:

$$\forall u, v \in Q \quad \text{with index}(u) < \text{index}(v) \implies d[u] \leq d[v] \leq d[u] + 1 \tag{1}$$

Consequently, when vertex v is first dequeued, its recorded distance $d[v]$ is guaranteed to be the exact geodesic distance $\delta(s, v)$. The runtime is bounded strictly by $O(|V| + |E|)$.

1.2 The Bounded Integer Extension: Edge Splitting ("Another BFS")

Consider the scenario where edge weights are non-uniform but restricted to small positive integers, specifically $w(e) \in \{1, 2\}$ for all $e \in E$. A natural question arises: Can we compute single-source shortest paths without the logarithmic overhead of a priority queue?

The answer is affirmative via the Edge Splitting Transformation (referred to pedagogically as "Another BFS"):

TABLE 1
Taxonomy of Single-Source Shortest Path (SSSP) Traversal Strategies

Algorithm	Edge Weight Restrictions	Primary Data Structure	Time Complexity	Space Complexity
Standard BFS	Uniform unweighted ($w(e) = 1$)	FIFO Queue (ArrayDeque)	$O(V + E)$	$O(V)$
Dummy-Node BFS	Small integers ($w(e) \in \{1, 2\}$)	FIFO Queue + Graph Expansion	$O(V + E)$	$O(V + E)$
0-1 BFS	Binary weights ($w(e) \in \{0, 1\}$)	Double-Ended Queue (Deque)	$O(V + E)$	$O(V)$
Dial's Algorithm	Small bounded integer $W = \max w(e)$	Array of Buckets $[0 \dots W \cdot V]$	$O(V \cdot W + E)$	$O(V + W)$
Dijkstra (Standard)	Non-negative weights ($w(e) \geq 0$)	Min-Heap (PriorityQueue)	$\mathbf{O((V + E) \log V)}$	$\mathbf{O(V + E)}$
Dijkstra (Fibonacci)	Non-negative weights ($w(e) \geq 0$)	Fibonacci Heap	$O(V \log V + E)$	$O(V)$
Bellman-Ford	Arbitrary weights (Negative edges allowed)	Iterative Edge Array	$O(V \cdot E)$	$O(V)$
SPFA (Shortest Path Faster)	Arbitrary weights (Negative edges allowed)	Queue-Optimized Relaxation	$O(V \cdot E)$ worst, $O(E)$ avg	$O(V)$

Definition 1 (Edge-Splitting Transformation). Given an edge $e = (u, v)$ with weight $w(u, v) = 2$, introduce an auxiliary dummy vertex $d_{new} \notin V$. Replace e with two directed unit edges:

$$e_1 = (u, d_{new}), \quad w(e_1) = 1 \quad \text{and} \quad e_2 = (d_{new}, v), \quad w(e_2) = 1 \tag{2}$$

For an undirected edge, two unit bidirectional edge pairs are created: $(u \leftrightarrow d_{new})$ and $(d_{new} \leftrightarrow v)$.

Proposition 2 (Complexity Bounds of Edge Splitting for $w \in \{1, 2\}$). Let $E_2 \subseteq E$ denote the set of edges having weight $w(e) = 2$, where $|E_2| \leq |E|$. The transformed graph $G' = (V', E')$ satisfies:

$$|V'| = |V| + |E_2| \leq |V| + |E| \tag{3}$$

$$|E'| = (|E| - |E_2|) + 2|E_2| = |E| + |E_2| \leq 2|E| \tag{4}$$

Running standard FIFO BFS on G' computes exact shortest paths in:

$$T(V, E) = O(|V'| + |E'|) = O((|V| + |E|) + 2|E|) = O(|V| + |E|) \tag{5}$$

with auxiliary space $S(V, E) = O(|V| + |E|)$.

This achieves linear time complexity, completely circumventing priority queue overhead.

1.3 The Scaling Breakdown: Why Arbitrary Weights Demand Priority Queues

While edge-splitting is optimal for $w \in \{1, 2\}$, it suffers from catastrophic degradation when edge weights become arbitrary integers $w(e) \in [1, W]$. If an edge possesses weight W, replacing it with unit-cost sub-segments requires the insertion of $W - 1$ intermediate dummy vertices:

$$u \xrightarrow{1} d_1 \xrightarrow{1} d_2 \xrightarrow{1} \dots \xrightarrow{1} d_{W-1} \xrightarrow{1} v \tag{6}$$

In practical applications and competitive benchmarks where $W \in [10^5, 10^9]$:

$$|V'| = |V| + \sum_{e \in E} (w(e) - 1) = O(|V| + |E| \cdot W) \tag{7}$$

$$|E'| = \sum_{e \in E} w(e) = O(|E| \cdot W) \tag{8}$$

For a graph with $|V| = 10^4$, $|E| = 5 \times 10^4$, and maximum edge weight $W = 10^5$, the dummy node expansion generates $|V'| \approx 5 \times 10^9$ vertices, resulting in immediate Out-Of-Memory (OOM) heap exhaustion and prohibitive Time Limit Exceeded (TLE) penalties.

Table 1 contextualizes this trade-off. To evaluate arbitrary non-negative weights efficiently, we must transition from static topological time slicing (uniform queue steps) to dynamic, priority-driven exploration governed by Dijkstra's Algorithm.

2 Dijkstra's Algorithm: Theoretical Foundations and Inductive Proof

Dijkstra's algorithm, formulated by Edsger W. Dijkstra in 1956, solves the Single-Source Shortest Path (SSSP) problem on weighted graphs with non-negative edge weights.

2.1 Formal Problem Formulation

Let $G = (V, E)$ be a directed or undirected graph equipped with a weight function $w : E \to \mathbb{R}_{\geq 0}$. A path P from source s to vertex v is a sequence of vertices $P = \langle v_0, v_1, \dots, v_k \rangle$ where $v_0 = s$, $v_k = v$, and $(v_{i-1}, v_i) \in E$ for all $1 \leq i \leq k$. The weight of path P is:

$$w(P) = \sum_{i=1}^{k} w(v_{i-1}, v_i) \tag{9}$$

The shortest-path distance $\delta(s, v)$ from s to v is defined as:

$$\delta(s, v) = \begin{cases} \min\{w(P) : s \xrightarrow{P} v\}, & \text{if a path exists} \\ \infty, & \text{otherwise} \end{cases} \tag{10}$$

2.2 The Relaxation Primitive

Dijkstra's algorithm maintains a distance array $d[v]$ representing the current upper bound on the shortest path distance from s to v, and a predecessor pointer $\pi[v]$ for path reconstruction. The fundamental operation is edge relaxation:

Definition 3 (Edge Relaxation). For an edge $(u, v) \in E$ with weight $w(u, v)$, relaxing (u, v) tests whether traversing through vertex u yields a shorter path to v:

$$\text{Relax}(u, v, w): \quad \begin{cases} \text{if } d[u] + w(u,v) < d[v] \implies & d[v] \leftarrow d[u] + w(u,v) \\ & \pi[v] \leftarrow u \end{cases} \tag{11}$$

2.3 Mathematical Proof of Correctness

The correctness of Dijkstra's algorithm hinges on the Greedy Choice Invariant: whenever a vertex u with the minimum tentative distance is extracted from the unvisited priority queue, its tentative distance is already exact, i.e., $d[u] = \delta(s, u)$.

Theorem 4 (Correctness of Dijkstra's Algorithm). Let $G = (V, E, w)$ with $w(e) \geq 0$ for all $e \in E$. At the moment any vertex $u \in V$ is extracted from the min-priority queue, $d[u] = \delta(s, u)$, and this value never subsequently changes.

Proof. We proceed by mathematical induction on the size of the set of finalized vertices $S \subseteq V$.

Base Case ($|S| = 1$): Initially, $S = \{s\}$. The source distance is initialized to $d[s] = 0$. Since edge weights are non-negative ($w(e) \geq 0$), no path from s to s can have total weight less than 0. Thus $d[s] = 0 = \delta(s, s)$. The base case holds.

Inductive Hypothesis: Assume that for all vertices $x \in S$, $d[x] = \delta(s, x)$.

Inductive Step: Consider the next vertex $u \in V \setminus S$ selected by the greedy criterion:

$$d[u] = \min_{z \in V \setminus S} d[z] \tag{12}$$

We must prove that $d[u] = \delta(s, u)$. Suppose, for the sake of contradiction, that $d[u] > \delta(s, u)$.

Since $d[u] > \delta(s, u)$, there must exist an actual shortest path P from s to u such that $w(P) < d[u]$. Because $s \in S$ and $u \in V \setminus S$, path P must at some point cross the cut $(S, V \setminus S)$.

Let (x, y) be the first edge on path P that exits S, such that $x \in S$ and $y \in V \setminus S$ (it is possible that $y = u$). Path P decomposes into:

$$s \xrightarrow{P_{s \to x}} x \xrightarrow{w(x,y)} y \xrightarrow{P_{y \to u}} u \tag{13}$$

By the inductive hypothesis, $x \in S \implies d[x] = \delta(s, x)$. When x was finalized, its outgoing edge (x, y) was relaxed, which ensured that:

$$d[y] \leq d[x] + w(x, y) = \delta(s, x) + w(x, y) = w(P_{s \to y}) \tag{14}$$

Because all edge weights are non-negative ($w(e) \geq 0$), the subpath $P_{y \to u}$ satisfies $w(P_{y \to u}) \geq 0$. Therefore:

$$w(P_{s \to y}) \leq w(P_{s \to y}) + w(P_{y \to u}) = w(P) = \delta(s, u) \tag{15}$$

Combining these inequalities:

$$d[y] \leq w(P_{s \to y}) \leq \delta(s, u) < d[u] \implies d[y] < d[u] \tag{16}$$

However, both y and u belong to $V \setminus S$. The algorithm chose u from $V \setminus S$ because u minimized tentative distance, which requires:

$$d[u] \leq d[y] \tag{17}$$

This produces an immediate logical contradiction:

$$d[u] \leq d[y] < d[u] \implies d[u] < d[u] \tag{18}$$

Hence, our initial assumption that $d[u] > \delta(s, u)$ is false. Because $d[u]$ is an upper bound on $\delta(s, u)$ maintained through valid relaxations, $d[u] \geq \delta(s, u)$. Thus $d[u] = \delta(s, u)$ identically. By induction, every finalized vertex satisfies $d[u] = \delta(s, u)$. □

2.4 Why Negative Edge Weights Break Dijkstra's Invariant

The inductive proof critically depends on inequality (12): $w(P_{y \to u}) \geq 0$. If an edge on $P_{y \to u}$ possesses a negative weight, it is possible that $w(P) < w(P_{s \to y})$, invalidating the monotonic expansion property.

Consider the 3-vertex directed graph depicted in Figure 1:

- Vertices: $V = \{1, 2, 3\}$, Source: $s = 1$.
- Edges: $(1, 2)$ with $w = 4$, $(1, 3)$ with $w = 2$, $(3, 2)$ with $w = -5$.

Execution trace:

1) $d[1] = 0$, $d[2] = \infty$, $d[3] = \infty$. Min-heap contains $(0, 1)$.
2) Pop $(0, 1)$: Relax edge $(1, 2) \implies d[2] = 4$; relax edge $(1, 3) \implies d[3] = 2$.
3) Min-heap contains $(2, 3)$ and $(4, 2)$. The greedy choice extracts $(2, 3)$ since $2 < 4$. Vertex 3 is marked finalized.

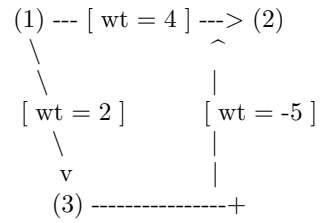


Fig. 1. Classic 3-vertex counterexample proving Dijkstra's failure with negative edge weights.

4) Relax edge $(3, 2) \implies d[2] = \min(4, 2 + (-5)) = \min(4, -3) = -3$.
5) Now vertex 2 is extracted with distance -3. But if vertex 2 had outgoing edges to other already-finalized nodes, those nodes would never be updated, leaving them with incorrect, suboptimal distances. If vertex 2 were extracted before vertex 3, it would be finalized at 4 and never updated to -3.

Thus, Dijkstra's algorithm cannot guarantee correctness in the presence of negative edge weights. When negative weights exist without negative cycles, the Bellman-Ford algorithm ($O(V \cdot E)$) or SPFA must be employed.

2.5 Dijkstra vs. Minimum Spanning Tree (MST - Prim's Algorithm)

A frequent point of conceptual confusion in systems design and technical interviews is conflating the Single-Source Shortest Path Tree (SPT) with a Minimum Spanning Tree (MST):

Definition 5 (Dijkstra SPT vs. Prim's MST).
- Dijkstra (SPT): Minimizes the path metric $\delta(s, v)$ from a single, designated source s to every individual vertex $v \in V$. The resulting tree minimizes individual root-to-leaf path lengths.
- Prim / Kruskal (MST): Minimizes the global cumulative edge weight $\sum_{e \in T} w(e)$ required to connect all vertices in G. It has no fixed source vertex.

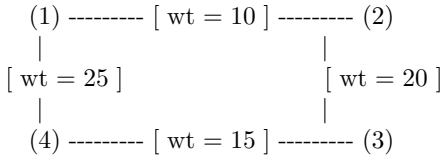


Fig. 2. Graph demonstrating the fundamental divergence between MST and Shortest Path Tree.

Consider the 4-vertex cyclic graph in Figure 2:

- Minimum Spanning Tree: Selects edges $(1, 2)$ [10], $(3, 4)$ [15], and $(2, 3)$ [20]. Total tree weight: $10 + 15 + 20 = 45$. In this tree, the path from 1 to 4 is $1 \to 2 \to 3 \to 4$, with total distance $10 + 20 + 15 = 45$.
- Dijkstra's SPT from Source 1: Selects edge $(1, 4)$ [25] directly, because $25 < 45$. The SPT consists of edges $(1, 2)$ [10], $(2, 3)$ [20], and $(1, 4)$ [25]. Total tree weight: $10 + 20 + 25 = 55$.

The shortest path from 1 to 4 has length 25, yet the MST forces a path length of 45 to minimize the global sum of weights ($45 < 55$). Hence, Dijkstra's algorithm cannot be used to find an MST, and an MST does not provide shortest paths.

3 Runtime Engineering: PriorityQueue Mechanics and Complexity

While theoretical algorithms assume an ideal priority queue supporting an $O(\log V)$ decreaseKey operation, real-world runtime libraries (including Java's java.util.PriorityQueue, C++'s std::priority_queue, and Python's heapq) do not provide an efficient indexed decreaseKey.

3.1 The decreaseKey Dilemma in Standard Runtimes

In an array-backed binary min-heap without an inverted index (mapping vertex ID $\to$ heap array slot), locating an arbitrary vertex v to decrease its key requires an $O(|Q|)$ linear search scan:

$$T_{\text{remove}}(v) = O(|Q|) \implies T_{\text{Dijkstra}} = O(V \cdot |Q| + E \cdot |Q|) = O(E \cdot V) \tag{19}$$

This degrades the algorithm to worse performance than naive array-based Dijkstra ($O(V^2)$).

To maintain logarithmic efficiency without implementing a complex custom Indexed Binary Heap, production software engineering employs the Lazy Deletion (Duplicate Insertion) Pattern.

3.2 The Lazy Deletion Pattern and the Stale Pruning Guard

Instead of updating an existing vertex's key in-place, whenever an edge relaxation discovers a strictly shorter path ($d_{\text{new}} < d[v]$), we immediately push a new state pair (d_{new}, v) into the priority queue:

```
if (newDist < dist[neigh]) {
    dist[neigh] = newDist;
    minHeap.offer(new int[]{newDist, neigh});
}
```

Because older, inferior entries (d_{old}, v) (where $d_{\text{old}} > d_{\text{new}}$) remain embedded in the heap, they will eventually rise to the top and be returned by poll().

Definition 6 (The Stale Pruning Guard). The stale pruning guard is an invariant check executed immediately upon extracting a state pair (d, u) from the priority queue:

```
int[] curr = minHeap.poll();
int d = curr[0];
int u = curr[1];

if (d > dist[u]) {
    continue; // STALE ENTRY: Discard immediately in O(1)
}
```

Why the guard is essential:

1) Preventing Redundant Edge Expansions: If a stale entry (d_{old}, u) is processed without this check, all $|Adj(u)|$ outgoing edges are re-traversed, multiplying work needlessly.
2) Preventing Algorithmic Blowup: In dense graphs with multiple path relaxations per vertex, omitting the guard leads to exponential state re-evaluations, converting an $O(E \log V)$ algorithm into worst-case exponential execution time.

3.3 Rigorous Complexity Derivation

Theorem 7 (Asymptotic Bounds of Dijkstra with Lazy Deletion). On a graph $G = (V, E)$ with non-negative edge weights, Dijkstra's algorithm implemented with a binary min-heap and lazy deletion executes in $O((V + E) \log V)$ time and requires $O(V + E)$ auxiliary space.

Proof. Let us quantify the operations on the priority queue Q:

1) Number of Insertions (offer): The source vertex is inserted once. Each subsequent edge $(u, v) \in E$ can trigger at most one successful relaxation that inserts an element into Q. Thus, the total number of heap insertions is bounded by:
$$N_{\text{insert}} \leq 1 + |E| = O(|E|) \tag{20}$$
2) Maximum Heap Capacity: Because each insertion adds one entry and elements are only removed upon extraction:
$$|Q| \leq |E| \tag{21}$$
For a simple graph, $|E| \leq |V|^2$. Therefore:
$$\log |Q| \leq \log(|V|^2) = 2 \log |V| = O(\log |V|) \tag{22}$$
3) Cost of Extraction (poll): Every element inserted into Q is extracted at most once. Hence, there are at most $O(|E|)$ poll() invocations. Each extraction on a heap of size $|Q|$ incurs:
$$T_{\text{poll}} = O(\log |Q|) = O(\log |V|) \tag{23}$$
Total extraction work: $O(|E| \log |V|)$.
4) Cost of Successful Vertex Finalizations: Exactly $|V|$ vertices satisfy the condition $d == dist[u]$ and proceed to examine their adjacency lists. For each finalized vertex u, all outgoing edges $(u, v) \in Adj(u)$ are evaluated once:
$$\sum_{u \in V} |Adj(u)| = |E| \tag{24}$$
5) Total Time Complexity:
$$\begin{aligned} T(V, E) &= O(|V|) + N_{\text{insert}} \cdot O(\log |V|) + N_{\text{extract}} \cdot O(\log |V|) + \sum_{u \in V} |Adj(u)| \\ &= O(|V|) + O(|E| \log |V|) + O(|E| \log |V|) + O(|E|) \\ &= O((|V| + |E|) \log |V|) \end{aligned} \tag{25}$$

For any connected graph where $|E| \geq |V| - 1$, this simplifies to:

$$T(V, E) = O(|E| \log |V|) \tag{26}$$

6) Space Complexity: The adjacency list requires $O(|V| + |E|)$. The distance array requires $O(|V|)$. The priority queue holds at most $|E|$ stale and active records concurrently. Thus:

$$S(V, E) = O(|V| + |E|) \tag{27}$$

□

4 Topological Sorting: Formulations, Invariants, and Existence

While Dijkstra structures search according to path cost accumulation, dependency scheduling requires establishing an ordering governed purely by directional precedence.

Definition 8 (Topological Ordering). Given a directed graph $G = (V, E)$, a topological sort is a linear ordering of vertices $L = \langle v_1, v_2, \ldots, v_{|V|} \rangle$ such that for every directed edge $(u, v) \in E$, vertex u strictly precedes vertex v in L:

$$(u, v) \in E \implies \text{pos}(u) < \text{pos}(v) \tag{28}$$

where $\text{pos}(x)$ denotes the index of vertex x in the linear sequence L.

4.1 The Fundamental Existence Theorem

Theorem 9 (DAG-Topological Sort Equivalence). A directed graph $G = (V, E)$ admits a topological sorting if and only if G is acyclic (i.e., G is a Directed Acyclic Graph, or DAG).

Proof. Necessity ($\implies$): Suppose G admits a topological ordering L. Assume for contradiction that G contains a directed cycle $C = \langle u_1, u_2, \ldots, u_k, u_1 \rangle$ of length $k \geq 2$. By definition of a topological sort:

$$(u_1, u_2) \in E \implies \text{pos}(u_1) < \text{pos}(u_2) \tag{29}$$
$$(u_2, u_3) \in E \implies \text{pos}(u_2) < \text{pos}(u_3) \tag{30}$$
$$\vdots$$
$$(u_{k-1}, u_k) \in E \implies \text{pos}(u_{k-1}) < \text{pos}(u_k) \tag{31}$$
$$(u_k, u_1) \in E \implies \text{pos}(u_k) < \text{pos}(u_1) \tag{32}$$

By transitivity of the strict ordering relation $<$ on integers:

$$\text{pos}(u_1) < \text{pos}(u_2) < \cdots < \text{pos}(u_k) < \text{pos}(u_1) \implies \text{pos}(u_1) < \text{pos}(u_1) \tag{33}$$

This is impossible by irreflexivity ($x \not< x$). Hence, G cannot contain any directed cycles; G must be acyclic.

Sufficiency ($\impliedby$): Suppose G is a finite directed acyclic graph. We first establish a critical lemma:

Lemma 10. Every finite directed acyclic graph G contains at least one vertex with in-degree 0 (a source) and at least one vertex with out-degree 0 (a sink).

Proof of Lemma. Pick an arbitrary vertex $v_1 \in V$. If v_1 has no incoming edges, in-degree is 0. Otherwise, select an incoming neighbor v_2 such that $(v_2, v_1) \in E$. If v_2 has no incoming edges, done; otherwise choose v_3 such that $(v_3, v_2) \in E$. Repeat this walk backwards. Because V is finite, by the Pigeonhole Principle, if this walk exceeds $|V|$ steps, some vertex must repeat, forming a directed cycle. But G is acyclic; hence, no vertex can repeat. The backward walk must terminate at a vertex v^* with no incoming edges. Thus $\text{indegree}(v^*) = 0$. (An identical symmetric argument walking forward proves the existence of a sink). □

We construct the topological order inductively: Select a vertex u with $\text{indegree}(u) = 0$. Place u as the next element in L. Remove u and all its outgoing edges from G. The remaining induced subgraph $G' = G \setminus \{u\}$ remains a finite DAG. By induction on $|V|$, G' has a valid topological sort L'. Concatenating u before L' produces a valid topological sort for G. □

4.2 Uniqueness Criterion and Hamiltonian Paths

A DAG frequently admits multiple valid topological orderings. For instance, if two independent course prerequisites have no mutual path, their relative order is arbitrary.

Theorem 11 (Uniqueness of Topological Ordering). A DAG $G = (V, E)$ possesses a unique topological ordering if and only if G contains a directed Hamiltonian path (a directed path visiting every vertex in V exactly once).

Proof. Let $L = \langle v_1, v_2, \ldots, v_n \rangle$ be a topological ordering. If $(v_i, v_{i+1}) \in E$ for all $1 \leq i \leq n-1$, then v_i must precede v_{i+1} in every valid ordering. Because this holds for every adjacent pair, no adjacent vertices can ever be swapped without violating an edge constraint. Hence L is uniquely determined.

Conversely, if for some index i, $(v_i, v_{i+1}) \notin E$, and there exists no directed path from v_i to v_{i+1}, then swapping v_i and v_{i+1} produces another valid topological ordering:

$$L' = \langle v_1, \ldots, v_{i-1}, v_{i+1}, v_i, v_{i+2}, \ldots, v_n \rangle \tag{34}$$

which satisfies all edge constraints. Thus, uniqueness requires an edge between every consecutively ordered pair, forming a Hamiltonian path. □

5 Algorithmic Duality: Kahn's Algorithm vs. Post-Order DFS

Two principal paradigms exist to compute topological orderings and detect cycles: Kahn's Algorithm (a BFS in-degree reduction method) and Tarjan's DFS Post-Order Reversal. They represent a fundamental algorithmic duality.

5.1 Kahn's Algorithm (In-Degree Reduction BFS)

Arthur Kahn (1962) proposed an iterative, source-driven elimination strategy:

1) In-Degree Computation: Calculate indegree$[v] = |\{u \in V : (u, v) \in E\}|$ for all $v \in V$ in $O(V + E)$ time.
2) Queue Seeding: Enqueue all vertices having indegree$[v] = 0$. These vertices have no unresolved dependencies.
3) Iterative Elimination: While the queue is non-empty:
   - Dequeue vertex u and append u to the topological order list.
   - For every outgoing neighbor $v \in Adj(u)$, decrement tentative in-degree: indegree$[v] \leftarrow$ indegree$[v] - 1$.
   - If indegree$[v]$ reaches 0, enqueue v.
4) Cycle Detection Invariant: Upon queue exhaustion, if $|Order| < |V|$, the graph contains one or more directed cycles!

Theorem 12 (Kahn's Cycle Detection Criterion). Kahn's algorithm processes all $|V|$ vertices if and only if G is acyclic. If $|Order| < |V|$, every vertex remaining with indegree$[v] > 0$ belongs to or is reachable from a directed cycle.

Proof. Vertices participating in a directed cycle C all possess at least one incoming edge within C. Their in-degrees can never drop to 0 because no vertex in C can be dequeued until another vertex in C is dequeued first—a circular lock. Consequently, no vertex in C is ever pushed into the queue. The queue terminates prematurely, leaving $|Order| < |V|$. □

5.2 DFS Post-Order Reversal (Tarjan's Formulation)

Depth-First Search traverses paths deeply until reaching a sink (a vertex with no outgoing unexplored edges), backtracking only when exploration along a branch completes.

Definition 13 (Post-Order Finish-Time Property). In a DAG, for any directed edge $(u, v) \in E$, the DFS finish time $f[u]$ of the source vertex u is strictly greater than the finish time $f[v]$ of the target vertex v:

$$(u, v) \in E \implies f[u] > f[v] \tag{35}$$

Therefore, appending each vertex to a list upon its DFS return (post-order) produces an ordering sorted in ascending finish time. Reversing this list yields a valid topological sort.

5.3 Three-Color State Cycle Detection in DFS

Unlike undirected graphs where cycle detection only requires a boolean visited[] array and a parent pointer, directed graphs require distinguishing between:

- Cross/Forward Edges: Edges pointing to vertices in completely different subtrees that have already finished processing. These do not indicate cycles.
- Back Edges: Edges pointing to ancestor vertices currently active on the recursive call stack. These do indicate cycles!

This is formalized using the classical **Three-Color Model**:

$$\text{State}[u] = \begin{cases} 0 \text{ (WHITE / UNVISITED):} & \text{Vertex has not yet been discovered.} \\ 1 \text{ (GRAY / VISITING):} & \text{Vertex is currently on the active recursion call stack.} \\ 2 \text{ (BLACK / VISITED):} & \text{Vertex and all descendants are fully explored and finalized.} \end{cases} \tag{36}$$

TABLE 2
Comprehensive Comparison: Kahn's In-Degree BFS vs. Post-Order DFS

Feature / Dimension	Kahn's Algorithm (BFS)	Post-Order DFS (Tarjan)
Core Mental Model	Pick courses with no prerequisites (Sources first)	Pick courses that are not prerequisites of others (Sinks first)
Primary Data Structures	FIFO Queue (ArrayDeque) + In-degree array	Recursion Call Stack + State array
Direction of Construction	Natural forward order ($v_1 \to v_2 \to \cdots \to v_n$)	Reverse order (appends sinks first, then reverses)
Cycle Detection Method	Count check: $\lvert Order\rvert \neq \lvert V\rvert$	Back-edge check: encountering State == VISITING
Lexicographical Ordering	Replace FIFO Queue with PriorityQueue	Complex; requires reverse post-order on sorted adjacencies
Recursion Depth Limit	Completely iterative; immune to call-stack overflow	Susceptible to StackOverflowError on deep DAGs ($V > 10^5$)
Time Complexity	$O(V+E)$	$O(V+E)$
Auxiliary Space	$O(V)$	$O(V)$

TABLE 3
Complete Step-by-Step Priority Queue and Distance Array Trace for 7-City Graph (Slide 6)

Step	Popped (d,u)	Status / Action	Edges Relaxed & Heap Insertions	Distance Array $d[1\ldots7]$	Priority Queue Contents
0	—	Init source $s=1$	Push $(0,1)$	$[0,\infty,\infty,\infty,\infty,\infty,\infty]$	$[(0,1)]$
1	$(0,1)$	Finalize 1	Relax $(1,2)\to 7$, $(1,3)\to 8$	$[0,7,8,\infty,\infty,\infty,\infty]$	$[(7,2),(8,3)]$
2	$(7,2)$	Finalize 2	Relax $(2,3)\to 10$ (>8, no-op); Relax $(2,4)\to 7+6=13$	$[0,7,8,13,\infty,\infty,\infty]$	$[(8,3),(10,3)^*,(13,4)]$
3	$(8,3)$	Finalize 3	Relax $(3,4)\to 8+4=12$ (<13); Relax $(3,5)\to 8+3=11$	$[0,7,8,12,11,\infty,\infty]$	$[(10,3)^*,(11,5),(12,4),(13,4)^*]$
4	$(\mathbf{10},\mathbf{3})$	STALE! ($10>d[3]=8$)	DISCARD IMMEDIATELY (Quiz Slide 7!)	$[0,7,8,12,11,\infty,\infty]$	$[(11,5),(12,4),(13,4)^*]$
5	$(11,5)$	Finalize 5	Relax $(5,7)\to 11+2=13$; Relax $(5,6)\to 11+5=16$; Relax $(5,4)\to 11+2=13$ (>12, no-op)	$[0,7,8,12,11,16,13]$	$[(12,4),(13,7),(13,4)^*,(16,6)]$
6	$(12,4)$	Finalize 4	Relax $(4,6)\to 12+5=17$ (>16, no-op)	$[0,7,8,12,11,16,13]$	$[(13,7),(13,4)^*,(16,6)]$
7	$(13,7)$	Finalize 7	Relax $(7,6)\to 13+2=15$ (<16);	$[0,7,8,12,11,\mathbf{15},13]$	$[(13,4)^*,(15,6),(16,6)^*]$
8	$(13,4)$	STALE! ($13>d[4]=12$)	DISCARD IMMEDIATELY	$[0,7,8,12,11,15,13]$	$[(15,6),(16,6)^*]$
9	$(15,6)$	Finalize 6	All adjacent finalized	$[0,7,8,12,11,15,13]$	$[(16,6)^*]$
10	$(16,6)$	STALE! ($16>d[6]=15$)	DISCARD IMMEDIATELY	$[0,7,8,12,11,15,13]$	[ ] (Empty)

* Denotes duplicate stale heap entries correctly pruned by the if (d > dist[u]) continue; guard.

Theorem 14 (DFS Back-Edge Cycle Invariant). A directed graph G contains a directed cycle if and only if a DFS traversal discovers an edge (u,v) such that State$[v] == 1$ (GRAY).

Table 2 highlights the trade-offs. Kahn's algorithm is preferred in production systems because it is fully iterative (avoiding JVM stack overflow), naturally supports tie-breaking priority queues, and identifies cycles through a simple count comparison.

6 Detailed Mathematical Walkthrough of Lecture Slide Traces

To ensure complete alignment with the lecture slides, we now trace and formally analyze every example, heap transition, and cycle scenario presented in the lecture deck.

6.1 Dijkstra Walkthrough 1: The 7-City Network (Slide 6 & 7)

Consider the 7-node country road network from Slide 6:

- Vertices: $V=\{1,2,3,4,5,6,7\}$, Source: $s=1$.
- Undirected Weighted Edges:

$$(1,2):7,\quad (1,3):8,\quad (2,3):3,\quad (2,4):6,$$
$$(3,4):4,\quad (3,5):3,\quad (4,5):2,\quad (4,6):5,$$
$$(5,7):2,\quad (7,6):2,\quad (5,6):5$$

6.1.1 Analysis of Quiz Slide 7

In Slide 7, the quiz asks:

> The current states are: min heap $=[(10,3),(13,4),(11,5),(12,4)]$, distance array $d=[0,7,8,\infty,\infty,\infty,\infty]$. What would be the next step to take place?

Correct Option: B ("Pick $(10,3)$ and Don't update the distance array").
Formal Rationale: The top of the min-heap is $(10,3)$. The current recorded shortest distance for vertex 3 is $d[3]=8$. Since the proposed distance $10>8$, this entry is a stale artifact from the earlier path $1\to 2\to 3$ (cost 10). Because vertex 3 was already reached via the shorter direct edge $1\to 3$ (cost 8), no update occurs, and the entry is safely discarded.

6.2 Dijkstra Walkthrough 2: The 5-Node Graph (Slide 23 Dry Run)

Consider the second dry run from Slide 23:

- Vertices: $V = \{1, 2, 3, 4, 5\}$, Source: $s = 1$.
- Directed Weighted Edges:

$$(1,2) : 10, \quad (1,3) : 5, \quad (3,2) : 2, \quad (3,4) : 15, \quad (2,5) : 5$$

TABLE 4
Trace of Dijkstra Dry Run (Slide 23)

Step	Popped (d, u)	Status / Action	Relaxations	Distance Array $d[1 \dots 5]$
0	—	Init source $s = 1$	Push $(0, 1)$	$[0, \infty, \infty, \infty, \infty]$
1	$(0, 1)$	Finalize 1	Relax $(1, 2) \to 10$; Relax $(1, 3) \to 5$	$[0, 10, 5, \infty, \infty]$
2	$(5, 3)$	Finalize 3	Relax $(3, 2) \to 5 + 2 = 7 < 10 \implies d[2] = 7$; Relax $(3, 4) \to 5 + 15 = 20 \implies d[4] = 20$	$[0, \mathbf{7}, 5, 20, \infty]$
3	$(7, 2)$	Finalize 2	Relax $(2, 5) \to 7 + 5 = 12 \implies d[5] = 12$	$[0, 7, 5, 20, \mathbf{12}]$
4	$(10, 2)$	STALE! $(10 > 7)$	Skip via Pruning Guard	$[0, 7, 5, 20, 12]$
5	$(12, 5)$	Finalize 5	No outgoing edges	$[0, 7, 5, 20, 12]$
6	$(20, 4)$	Finalize 4	No outgoing edges	$[0, 7, 5, 20, 12]$

Final shortest distance vector: $d = [0, 7, 5, 20, 12]$. This matches Slide 23.

6.3 Topological Sort Walkthrough 1: Kahn's Algorithm on 7-Node DAG (Slide 14)

From Slide 14, consider the DAG with vertices $\{1, 2, 3, 4, 5, 6, 7\}$ and edges:

$$\begin{aligned} &1 \to 2, \quad 1 \to 4 \\ &2 \to 3, \quad 2 \to 4 \\ &3 \to 5, \quad 3 \to 6 \\ &4 \to 5 \\ &5 \to 6 \\ &7 \to 6 \end{aligned}$$

1) In-Degree Calculation:

$$\begin{aligned} &\text{indeg}[1] = 0, \quad \text{indeg}[7] = 0 \\ &\text{indeg}[2] = 1 \text{ (from 1)} \\ &\text{indeg}[3] = 1 \text{ (from 2)} \\ &\text{indeg}[4] = 2 \text{ (from 1, 2)} \\ &\text{indeg}[5] = 2 \text{ (from 3, 4)} \\ &\text{indeg}[6] = 3 \text{ (from 3, 5, 7)} \end{aligned}$$

2) Initial Queue: Vertices with in-degree 0: $Q = [1, 7]$.
3) Dequeue 1: Append 1. Decrement indeg[2] ($1 \to 0$, enqueue 2) and indeg[4] ($2 \to 1$). $Q = [7, 2]$.
4) Dequeue 7: Append 7. Decrement indeg[6] ($3 \to 2$). $Q = [2]$.
5) Dequeue 2: Append 2. Decrement indeg[3] ($1 \to 0$, enqueue 3) and indeg[4] ($1 \to 0$, enqueue 4). $Q = [3, 4]$ (or $[4, 3]$).
6) Dequeue 4: Append 4. Decrement indeg[5] ($2 \to 1$). $Q = [3]$.
7) Dequeue 3: Append 3. Decrement indeg[5] ($1 \to 0$, enqueue 5) and indeg[6] ($2 \to 1$). $Q = [5]$.
8) Dequeue 5: Append 5. Decrement indeg[6] ($1 \to 0$, enqueue 6). $Q = [6]$.
9) Dequeue 6: Append 6. Queue empty.

Resulting Topological Ordering: $\langle 1, 7, 2, 4, 3, 5, 6 \rangle$. Since $|Order| = 7 = |V|$, the ordering is complete and valid.

6.4 Topological Sort Walkthrough 2: Cycle Detection Failure (Slide 17)

Consider Slide 17, which models the impossibility of finishing prerequisites when a dependency cycle exists:

- Vertices: $\{1, 2, 3, 4\}$.
- Edges: $1 \to 2$, $2 \to 3$, $3 \to 4$, $4 \to 2$ (cycle $2 \to 3 \to 4 \to 2$).
- In-Degrees: indeg[1] = 0, indeg[2] = 2 (from 1 and 4), indeg[3] = 1, indeg[4] = 1.

Execution trace:

1) Initial queue: $Q = [1]$ (only vertex with in-degree 0).

2) Dequeue 1: Append 1 to $Order$. Decrement neighbor 2: indeg[2] $= 2 - 1 = 1$.
3) In-degree of 2 is $1 \neq 0$, so 2 is not enqueued.
4) Queue is now empty!
5) Loop terminates. $Order = [1]$.
6) Evaluated condition: $|Order| = 1 \neq |V| = 4 \implies$ "Cycle Exists".

6.5 Topological Sort Walkthrough 3: DFS Post-Order Reversal (Slide 18)

From Slide 18, consider the 6-node DAG with edges:

$$1 \to 2, \quad 2 \to 3, \quad 3 \to 4, \quad 1 \to 6, \quad 6 \to 3, \quad 5 \to 6$$

Running DFS starting from vertex 1:

$$\begin{aligned} &\text{DFS}(1) \to \text{DFS}(2) \to \text{DFS}(3) \to \text{DFS}(4) \\ &\quad \text{Vertex 4 has no neighbors} \implies \text{Order.add}(4) \\ &\quad \text{Backtrack to 3: all visited} \implies \text{Order.add}(3) \\ &\quad \text{Backtrack to 2: all visited} \implies \text{Order.add}(2) \\ &\quad \text{Backtrack to 1: branch to 6} \to \text{DFS}(6) \\ &\quad \text{From 6: neighbor 3 is already visited} \implies \text{Order.add}(6) \\ &\quad \text{Backtrack to 1: all visited} \implies \text{Order.add}(1) \end{aligned}$$

Next outer loop explores unvisited vertex 5:

$$\text{DFS}(5) \to \text{neighbor 6 already visited} \implies \text{Order.add}(5)$$

Accumulated Post-Order Sequence: $[4, 3, 2, 6, 1, 5]$.
Reversing this sequence yields: $\langle \mathbf{5}, \mathbf{1}, \mathbf{6}, \mathbf{2}, \mathbf{3}, \mathbf{4} \rangle$.
Checking every edge confirms validity:

$$5 \prec 6, \quad 1 \prec 2, \quad 1 \prec 6, \quad 6 \prec 3, \quad 2 \prec 3, \quad 3 \prec 4$$

6.6 Analysis of Quiz Slide 22

Quiz Slide 22 presents 4 task nodes: $\{T_A, T_B, T_C, T_D\}$ with dependencies:

$$T_D \to T_A, \quad T_D \to T_C, \quad T_A \to T_B, \quad T_A \to T_C, \quad T_C \to T_B$$

In-Degrees: indeg$[T_D] = 0$, indeg$[T_A] = 1$ (from T_D), indeg$[T_C] = 2$ (from T_D, T_A), indeg$[T_B] = 2$ (from T_A, T_C).
Processing:

- Only T_D has in-degree 0: First is T_D.
- Dequeue T_D: decrements indeg$[T_A]$ to 0, indeg$[T_C]$ to 1. Next is T_A.
- Dequeue T_A: decrements indeg$[T_C]$ to 0, indeg$[T_B]$ to 1. Next is T_C.
- Dequeue T_C: decrements indeg$[T_B]$ to 0. Next is T_B.

Unique Topological Order: $\langle \mathbf{T_D}, \mathbf{T_A}, \mathbf{T_C}, \mathbf{T_B} \rangle$ (Option A).

7 Production Java Reference Implementations

We provide battle-tested, production-grade Java implementations engineered for optimal performance, memory footprint, and interview clarity.

7.1 Small-Weight SSSP via Edge Splitting (Listing 1)

Listing 1 implements the exact edge-splitting expansion taught in Slides 2–4 for $w(e) \in \{1, 2\}$, maintaining strict $O(V + E)$ time and space bounds.

```
import java.util.*;

public class SmallWeightBfs {
    /**
     * Solves SSSP on connected graphs where edge weights are strictly 1 or 2.
     * Expands weight-2 edges with dummy intermediate nodes, achieving O(V + E).
     *
     * @param numVertices Original vertex count (1-indexed: 1 to numVertices)
     * @param edges       Array of undirected edges [u, v, weight]
     * @param src         Source vertex
     * @param dest        Destination vertex
```

```java
     * @return Minimum geodesic distance from src to dest
     */
    public static int findShortestPath(int numVertices, int[][] edges, int src, int dest) {
        int maxDummyNodes = edges.length;
        int totalNodes = numVertices + maxDummyNodes;

        List<List<Integer>> adj = new ArrayList<>(totalNodes + 1);
        for (int i = 0; i <= totalNodes; i++) {
            adj.add(new ArrayList<>());
        }

        int dummyNodeId = numVertices + 1;
        for (int[] edge : edges) {
            int u = edge[0];
            int v = edge[1];
            int wt = edge[2];

            if (wt == 1) {
                adj.get(u).add(v);
                adj.get(v).add(u);
            } else if (wt == 2) {
                int dummy = dummyNodeId++;
                // Split u <-> v into u <-> dummy and dummy <-> v
                adj.get(u).add(dummy);
                adj.get(dummy).add(u);

                adj.get(dummy).add(v);
                adj.get(v).add(dummy);
            } else {
                throw new IllegalArgumentException("Weight must be 1 or 2, got: " + wt);
            }
        }

        // Standard FIFO BFS on unweighted transformed graph
        int[] dist = new int[dummyNodeId];
        Arrays.fill(dist, -1);

        Queue<Integer> queue = new ArrayDeque<>();
        queue.offer(src);
        dist[src] = 0;

        while (!queue.isEmpty()) {
            int curr = queue.poll();
            if (curr == dest) {
                return dist[dest];
            }

            for (int neighbor : adj.get(curr)) {
                if (dist[neighbor] == -1) {
                    dist[neighbor] = dist[curr] + 1;
                    queue.offer(neighbor);
                }
            }
        }

        return dist[dest];
    }
}
```

Listing 1. Small-Weight SSSP via Dummy Node Expansion

7.2 Industrial Dijkstra with Lazy Deletion and Path Reconstruction (Listing 2)

Listing 2 provides a complete, leak-free implementation of Dijkstra's algorithm. It incorporates the stale pruning guard, handles unreachable nodes, and reconstructs the optimal shortest path.

```java
import java.util.*;

public class DijkstraShortestPath {
    public static class Edge {
```

```
        public final int to;
        public final int weight;

        public Edge(int to, int weight) {
            this.to = to;
            this.weight = weight;
        }
    }

    public static class DijkstraResult {
        public final int[] distances;
        public final int[] parents;

        public DijkstraResult(int[] distances, int[] parents) {
            this.distances = distances;
            this.parents = parents;
        }

        public List<Integer> reconstructPath(int target) {
            if (distances[target] == Integer.MAX_VALUE) {
                return Collections.emptyList(); // Unreachable
            }
            List<Integer> path = new ArrayList<>();
            for (int at = target; at != -1; at = parents[at]) {
                path.add(at);
            }
            Collections.reverse(path);
            return path;
        }
    }

    /**
     * Computes Single-Source Shortest Paths in O((V + E) log V) using Lazy Deletion.
     *
     * @param numVertices Total vertices (1-indexed: 1 to numVertices)
     * @param adj         Adjacency list mapping u -> list of Edge(v, wt)
     * @param source      Source vertex
     * @return DijkstraResult containing distance array and parent pointers
     */
    public static DijkstraResult computeSSSP(int numVertices, List<List<Edge>> adj, int source)
        {
        int[] dist = new int[numVertices + 1];
        int[] parent = new int[numVertices + 1];
        Arrays.fill(dist, Integer.MAX_VALUE);
        Arrays.fill(parent, -1);

        // Min-heap ordered strictly by distance: int[] = {tentativeDist, vertexId}
        PriorityQueue<int[]> minHeap = new PriorityQueue<>(Comparator.comparingInt(a -> a[0]));

        dist[source] = 0;
        minHeap.offer(new int[]{0, source});

        while (!minHeap.isEmpty()) {
            int[] top = minHeap.poll();
            int currentDist = top[0];
            int u = top[1];

            // CRITICAL STALE PRUNING GUARD (O(1) skip)
            if (currentDist > dist[u]) {
                continue;
            }

            for (Edge edge : adj.get(u)) {
                int v = edge.to;
                int weight = edge.weight;

                if (dist[u] + weight < dist[v]) {
                    dist[v] = dist[u] + weight;
                    parent[v] = u;
                    minHeap.offer(new int[]{dist[v], v});
```

```
                }
            }
        }

        return new DijkstraResult(dist, parent);
    }
}
```

Listing 2. Production Dijkstra with Lazy Deletion and Path Reconstruction

7.3 Kahn's Algorithm with Cycle Detection (Listing 3)

Listing 3 implements Kahn's in-degree reduction algorithm, cleanly detecting cycles and returning an empty array if topological ordering is impossible.

```
import java.util.*;

public class KahnsTopologicalSort {
    /**
     * Computes topological order of a directed graph.
     *
     * @param numVertices Number of vertices (0 to numVertices - 1)
     * @param adj         Adjacency list
     * @return Valid topological order array, or empty array if cycle exists
     */
    public static int[] getTopologicalOrder(int numVertices, List<List<Integer>> adj) {
        int[] inDegree = new int[numVertices];
        for (int u = 0; u < numVertices; u++) {
            for (int v : adj.get(u)) {
                inDegree[v]++;
            }
        }

        Queue<Integer> queue = new ArrayDeque<>();
        for (int i = 0; i < numVertices; i++) {
            if (inDegree[i] == 0) {
                queue.offer(i);
            }
        }

        int[] order = new int[numVertices];
        int processedCount = 0;

        while (!queue.isEmpty()) {
            int u = queue.poll();
            order[processedCount++] = u;

            for (int v : adj.get(u)) {
                inDegree[v]--;
                if (inDegree[v] == 0) {
                    queue.offer(v);
                }
            }
        }

        // Cycle Detection Invariant: If count != numVertices, cycle exists!
        if (processedCount != numVertices) {
            return new int[0]; // Graph is cyclic
        }

        return order;
    }
}
```

Listing 3. Kahn's Algorithm for Topological Sort and Cycle Detection

7.4 Post-Order DFS with 3-Color State Cycle Detection (Listing 4)

Listing 4 implements recursive DFS topological sorting with the formal 3-color state machine (UNVISITED, VISITING, VISITED), ensuring robust cycle detection.

```java
import java.util.*;

public class DfsTopologicalSort {
    private static final int UNVISITED = 0;
    private static final int VISITING  = 1;
    private static final int VISITED   = 2;

    public static int[] getTopologicalOrder(int numVertices, List<List<Integer>> adj) {
        int[] state = new int[numVertices];
        List<Integer> postOrder = new ArrayList<>(numVertices);

        for (int i = 0; i < numVertices; i++) {
            if (state[i] == UNVISITED) {
                if (hasCycleDfs(i, adj, state, postOrder)) {
                    return new int[0]; // Cycle detected
                }
            }
        }

        // Post-order contains sinks first; reverse to obtain topological sort
        int[] result = new int[numVertices];
        for (int i = 0; i < numVertices; i++) {
            result[i] = postOrder.get(numVertices - 1 - i);
        }
        return result;
    }

    private static boolean hasCycleDfs(int u, List<List<Integer>> adj, int[] state, List<
        Integer> postOrder) {
        state[u] = VISITING; // Mark active on recursion stack

        for (int neighbor : adj.get(u)) {
            if (state[neighbor] == VISITING) {
                return true; // BACK-EDGE DETECTED: Cycle confirmed!
            }
            if (state[neighbor] == UNVISITED) {
                if (hasCycleDfs(neighbor, adj, state, postOrder)) {
                    return true;
                }
            }
        }

        state[u] = VISITED; // Mark finalized
        postOrder.add(u);   // Record upon finish time backtracking
        return false;
    }
}
```

Listing 4. DFS Topological Sort with 3-Color State Machine

8 Industrial and Competitive Programming Archetypes

We synthesize these principles through three canonical algorithmic archetypes frequently encountered in enterprise distributed systems and technical evaluations.

8.1 Course Schedule I & II (LeetCode 207 & 210)

Given N courses labeled 0 to $N-1$ and prerequisite pairs $[a, b]$ where course b must be taken before course a ($b \to a$), determine if all courses can be finished and return a valid sequence.

- Graph Construction: Directed edge $b \to a$.
- Resolution: Direct application of Kahn's Algorithm (Listing 3). If the processed order size equals N, return the sequence; otherwise return an empty array.

8.2 Network Delay Time (LeetCode 743)

Given a network of n nodes and directed transmission latencies $times[i] = (u, v, w)$, find the time required for a signal broadcast from node k to reach all nodes.

- Mathematical Formulation: Broadcast completion time is the maximum geodesic shortest path distance from source k:

$$T_{\text{complete}} = \max_{v \in V} \delta(k, v) \tag{37}$$

  If any vertex remains at ∞, return -1.
- Resolution: Direct application of Dijkstra's algorithm (Listing 2).

8.3 Alien Dictionary (LeetCode 269)

Given a sorted dictionary of words in an alien language, derive the lexicographical character ordering.

- Graph Construction: Compare adjacent words W_i and W_{i+1}. Find the first differing character $c_1 = W_i[k]$ and $c_2 = W_{i+1}[k]$. The precedence constraint dictates a directed edge $c_1 \to c_2$.
- Edge Case (Prefix Invalidation): If W_{i+1} is a strict prefix of W_i (e.g., "apple" before "app"), the dictionary is invalid; return empty string immediately.
- Resolution: Apply Kahn's Algorithm on unique characters. If a cycle is detected, no consistent alphabet exists.

9 Conclusion and Engineering Decision Framework

Choosing the correct graph traversal paradigm is an exercise in matching data structures to edge weight semantics and directional constraints:

1) Unweighted Graphs: Always employ standard FIFO BFS ($O(V + E)$).
2) Weights $\in \{1, 2\}$: Apply Edge Splitting with dummy nodes to retain linear $O(V + E)$ performance without priority queue overhead.
3) General Non-Negative Weights ($w \geq 0$): Employ Dijkstra's Algorithm with a binary min-heap and lazy deletion ($O((V + E) \log V)$). Always include the stale-element pruning guard if (d > dist[u]) continue;.
4) Dependency Precedence / DAGs: Employ Kahn's Algorithm for iterative safety and trivial cycle detection, or 3-color DFS for deep recursive traversal.

By adhering to this theoretical and practical foundation, software engineers can design resilient, mathematically sound, and asymptotically optimal graph algorithms.

References

[1] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, Introduction to Algorithms, 3rd ed. Cambridge, MA, USA: MIT Press, 2009.
[2] E. W. Dijkstra, "A note on two problems in connexion with graphs," Numerische Mathematik, vol. 1, no. 1, pp. 269–271, 1959.
[3] A. B. Kahn, "Topological sorting of large networks," Communications of the ACM, vol. 5, no. 11, pp. 558–562, 1962.
[4] R. Tarjan, "Depth-first search and linear graph algorithms," SIAM Journal on Computing, vol. 1, no. 2, pp. 146–160, 1972.
[5] R. Sedgewick and K. Wayne, Algorithms, 4th ed. Boston, MA, USA: Addison-Wesley Professional, 2011.